Day 5 — TCP Connection Lifecycle + Socket APIs

Today we connect the **TCP protocol** you learned on Day 4 with the **Linux socket APIs** you were asking about earlier.

The main goal is to understand:

```
socket()
   ↓
bind()
   ↓
listen()
   ↓
accept()

        SERVER

connect()
   ↓
TCP 3-way handshake
   ↓
ESTABLISHED
   ↓
send()/recv()
   ↓
close()
   ↓
TCP connection termination
```

---

1. First: What happens when a TCP server starts?

Suppose PC-B is a server.

```
PC-B
192.168.1.20
Port 8080
```

The server program does:

```
socket()
bind()
listen()
accept()
```

Let's understand each one.

---

2. `socket()` — Create a communication endpoint

```c
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

Think:

> "Linux, give me a TCP socket."

Parameters

```
socket(domain, type, protocol)
```

Parameter	Meaning
`AF_INET`	IPv4
`SOCK_STREAM`	TCP-style byte-stream socket
`0`	Let the system choose the appropriate protocol

Conceptually:

```
Application
    |
    | socket()
    ▼
Linux Kernel
    |
    ▼
TCP Socket created
```

Important:

`socket()` **does NOT establish a TCP connection.**

It creates the socket endpoint.

---

3. `bind()` — Give the socket a local address

Now the server wants:

```
IP   = 192.168.1.20
Port = 8080
```

It calls:

```
bind(sockfd, ...);
```

Think:

> "Kernel, associate this socket with this local IP/port."

Visual:

```
Server application
        |
        | bind()
        ▼
┌──────────────────────┐
| TCP Socket           |
|                      |
| Local IP: 192.168.1.20
| Local Port: 8080     |
└──────────────────────┘
```

Very important

`bind()` **does not connect to PC-A**.

It only establishes the **local address identity** of the socket.

---

4. `listen()` — Become a TCP listening socket

Server:

```
listen(sockfd, 10);
```

Now the server says:

> "I am ready to receive incoming TCP connection requests."

Conceptually:

```
Server

socket()
   ↓
bind()
   ↓
listen()
```

The socket is now a **listening socket**.

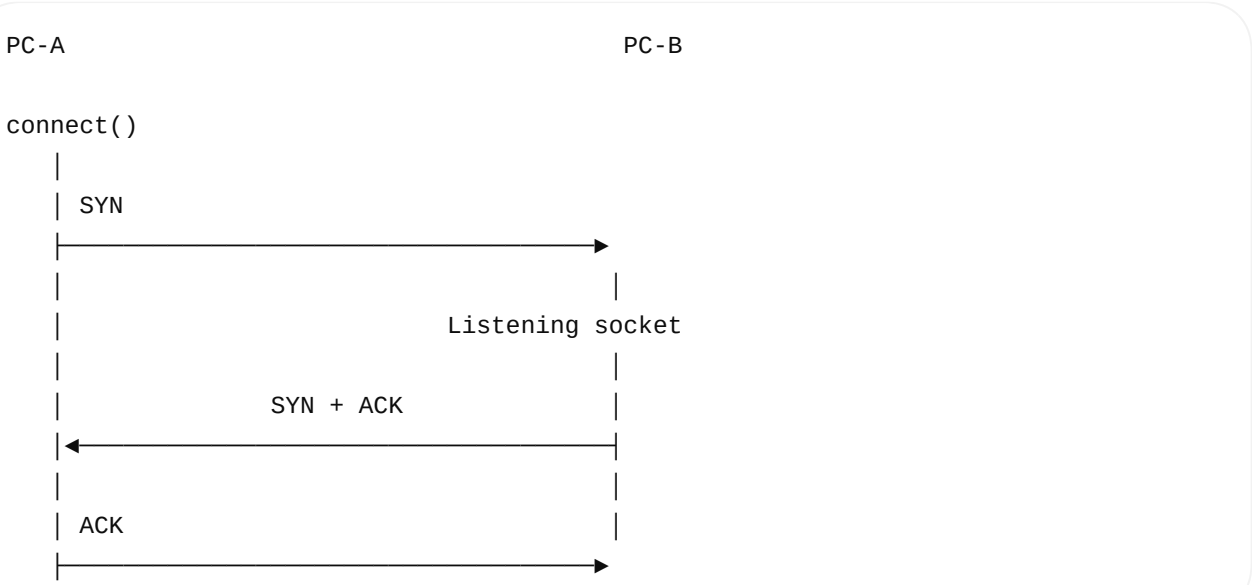
---

5. Client calls `connect()`

PC-A wants to connect.

```
connect(sockfd, ...);
```

Now TCP handshake starts.



After the handshake:

```
TCP connection = ESTABLISHED
```

---

6. Where does `accept()` fit?

This is a very important beginner question.

The server has:

```
listen(sockfd, 10);
```

Then:

```
int clientfd = accept(sockfd, ...);
```

Think of it like this:

```
                SERVER

          Listening Socket
                |
                |
        Incoming connection
                |
                ▼
          accept()
                |
                ▼
        Connected Socket
```

The original listening socket continues listening.

The returned socket is used for communication with that client.

---

7. Very Important Difference

Suppose:

```c
int serverfd = socket(...);

bind(serverfd, ...);

listen(serverfd, 10);

int clientfd = accept(serverfd, ...);
```

You now have:

```
serverfd
   ↓
Listening socket

clientfd
   ↓
Connected socket
```

They have different purposes.

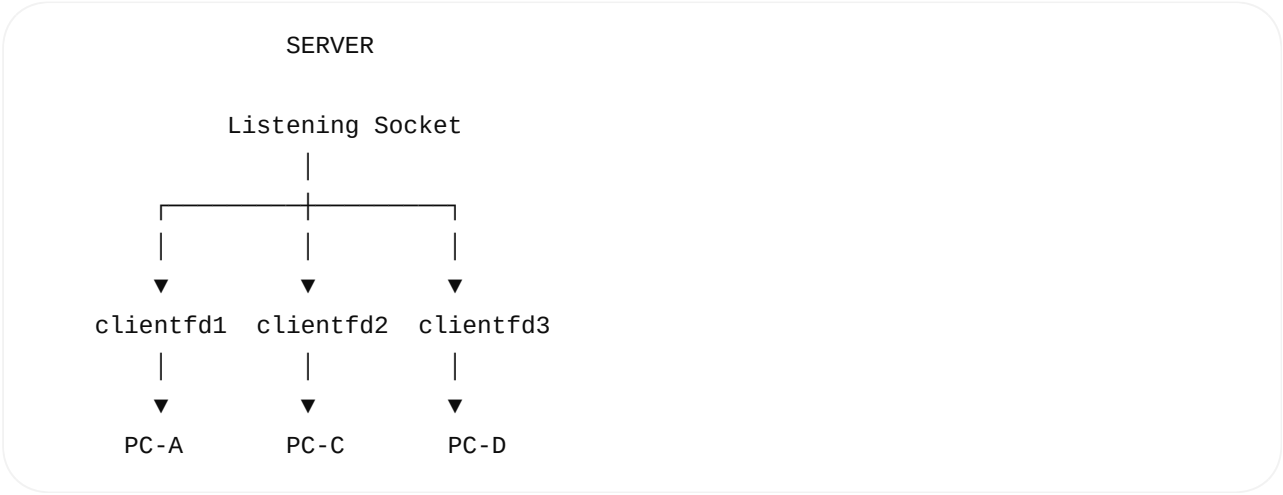
```
serverfd
   |
   └── Wait for new clients

clientfd
```

```
    |
    └── Communicate with this client
```

---

8. Multiple Clients

This becomes easier now.



The server can accept multiple connections.

Each connected client gets its own connected socket.

---

9. Complete Server Flow

```
socket()
   |
   ▼
Create socket
   |
   ▼
bind()
   |
   ▼
Assign local IP + port
   |
   ▼
listen()
   |
   ▼
Wait for clients
   |
   ▼
accept()
   |
   ▼
Connected socket
```

```
   |
   ▼
send()/recv()
   |
   ▼
close()
```

---

10. Complete Client Flow

Client is simpler:

```
socket()
   |
   ▼
Create socket
   |
   ▼
connect()
   |
   ▼
TCP handshake
   |
   ▼
ESTABLISHED
   |
   ▼
send()/recv()
   |
   ▼
close()
```

---

11. Now Connect This to TCP States

This is where **TCP state machines** become important.

The server:

```
CLOSED
   |
socket()
   ▼
CLOSED* / local socket setup
   |
bind()
   ▼
LISTEN
   |
```

```
   | incoming SYN
   ▼
SYN-RECEIVED
   |
   | handshake completed
   ▼
ESTABLISHED
```

The client:

```
CLOSED
   |
connect()
   ▼
SYN-SENT
   |
   | receives SYN+ACK
   ▼
ESTABLISHED
```

---

12. The Three Most Important States

For now focus on:

```
LISTEN
SYN-SENT
ESTABLISHED
```

`LISTEN`

Server is waiting for connection requests.

```
Server
   ↓
LISTEN
   ↓
"Anyone want to connect?"
```

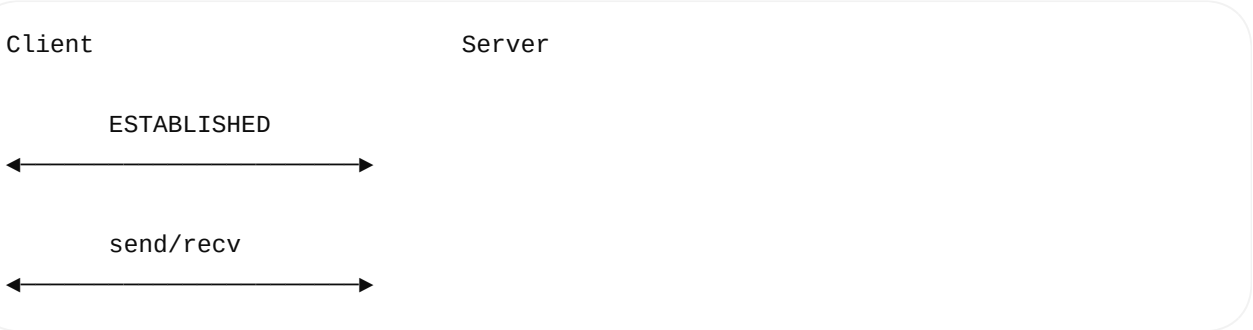
`SYN-SENT`

Client has sent SYN and is waiting for the response.

```
Client

SYN
──────►
```

`ESTABLISHED`

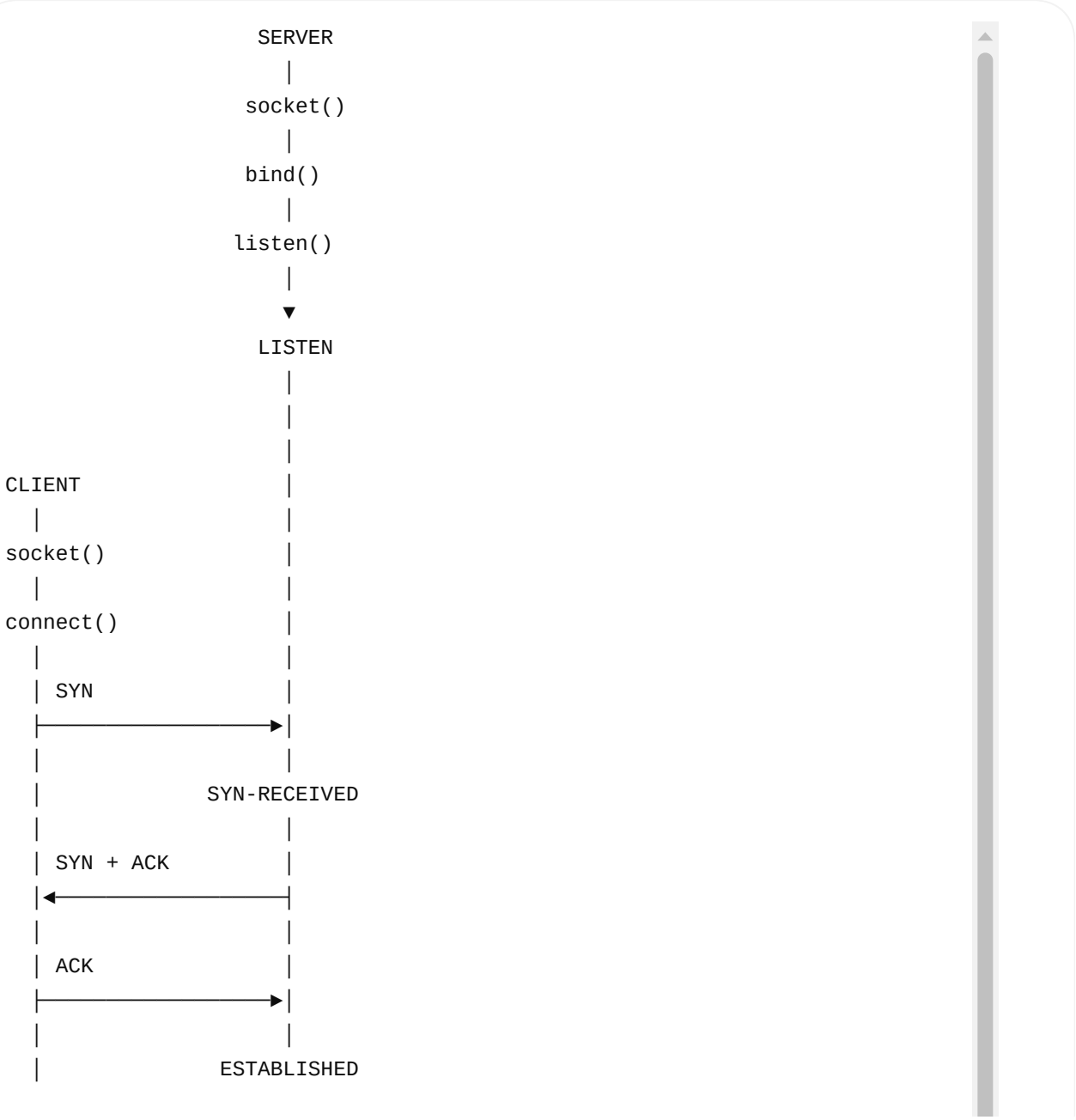
TCP connection is ready for normal data transfer.

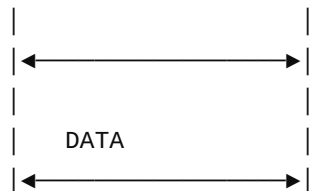


---

13. Full Handshake + Socket API Mapping

This is extremely useful for interviews.





---

14. Where Does `accept()` Happen?

After the TCP handshake has progressed sufficiently, the server's `accept()` can return a connected socket.

Conceptually:

```
listen()
   |
   ▼
LISTEN
   |
   | SYN arrives
   ▼
TCP processing
   |
   ▼
Connection established
   |
   ▼
accept()
   |
   ▼
connected socket
```

So don't think:

```
❌ accept() sends SYN
```

Instead:

```
connect()
     ↓
initiates client-side connection establishment

TCP kernel stack
     ↓
handles SYN/SYN+ACK/ACK

accept()
     ↓
application obtains the connected socket
```

15. Does `connect()` Directly "Connect to Transport Layer"?

This connects directly to your previous question.

When your application calls:

```
connect()
```

the system call enters the kernel.

Conceptually:

```
Application
     |
     | connect()
     ▼
Linux Kernel
     |
     ▼
TCP implementation
     |
     ▼
TCP SYN
     |
     ▼
IP
     |
     ▼
Network interface
```

So:

> **`connect()` is a socket API exposed to the application. The kernel's TCP implementation performs the TCP connection establishment.**

The application does not manually create the SYN packet.

---

16. What About `bind()`?

Similarly:

```
bind()
```

does not mean:

```
❌ TCP handshake
❌ SYN
❌ SYN+ACK
```

It means:

```
Application
    |
    | bind()
    ▼
Kernel
    |
    ▼
Associate socket with
local address/port
```

For example:

```
192.168.1.20:8080
```

---

17. What About `send()`?

After:

```
ESTABLISHED
```

application calls:

```
send(sockfd, data, length, 0);
```

Then:

```
Application
    |
    | send()
    ▼
Kernel
    |
    ▼
TCP
    |
    | segmentation
    ▼
TCP Segment
    |
    ▼
```

```
IP Packet
    |
    ▼
Ethernet Frame
    |
    ▼
NIC
```

---

18. TCP Connection Termination

Now let's understand `close()`.

Suppose PC-A is finished.

```
close(sockfd);
```

TCP doesn't simply disappear immediately.

TCP performs an orderly termination using **FIN** and **ACK**.

---

19. FIN

FIN = Finish

It means:

> "I have finished sending data."

Example:

```
PC-A                        PC-B

FIN
──────────────────────────►
```

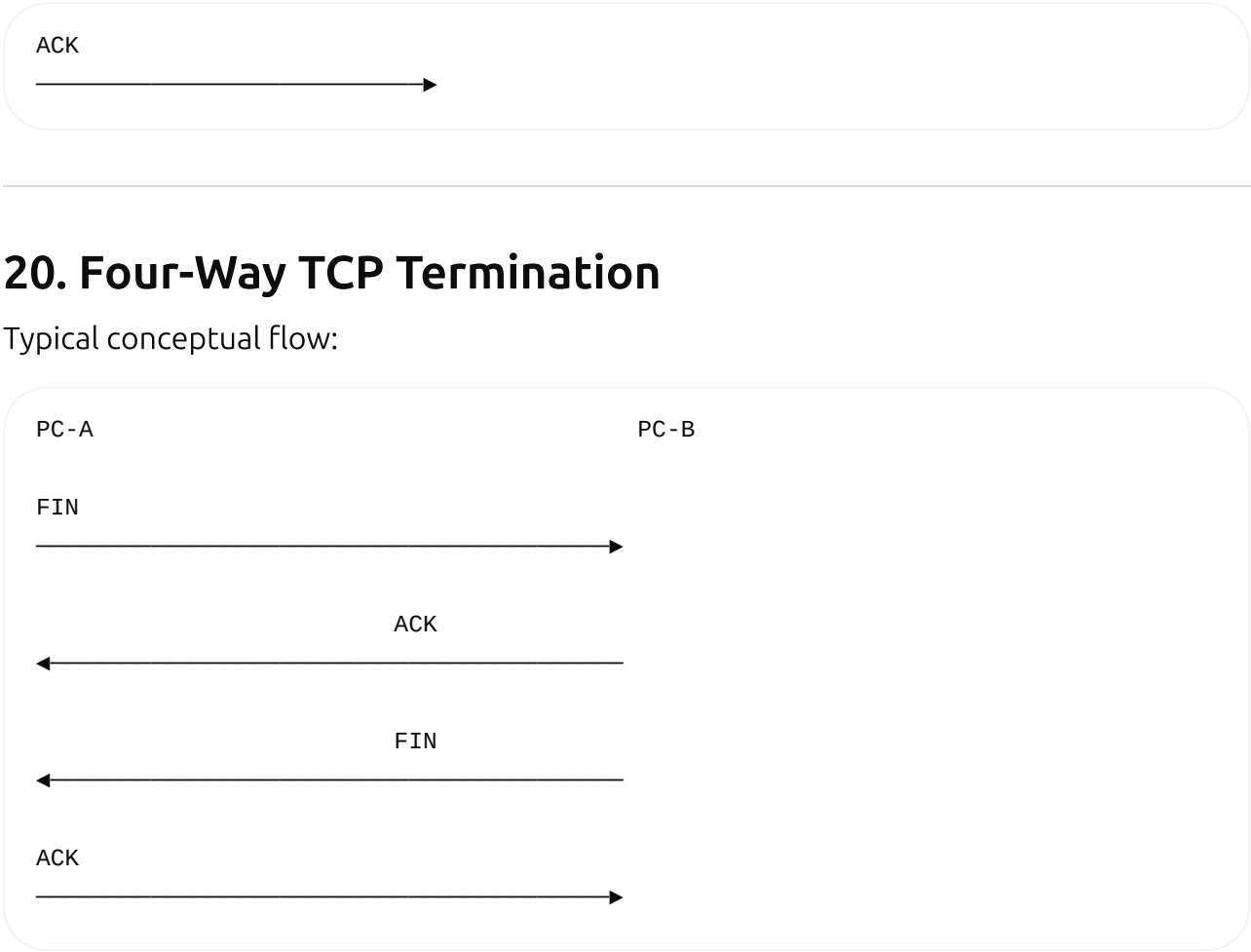
PC-B responds:

```
ACK
◄──────────────────────────
```

Then PC-B eventually sends its own FIN:

```
FIN
◄──────────────────────────
```

PC-A responds:



---

20. Four-Way TCP Termination

Typical conceptual flow:

Then the connection closes.

Why four messages?

Because TCP is **full-duplex**.

Each direction of data transmission can be closed independently.

---

21. Important TCP States During Close

You don't need to memorize every state today.

Know these first:

```
     ▼
CLOSED
```

The other side may go through:

```
ESTABLISHED
     |
     ▼
CLOSE-WAIT
     |
     ▼
LAST-ACK
     |
     ▼
CLOSED
```

---

22. Why TIME-WAIT?

This is a very common interview question.

After actively closing a TCP connection, the endpoint can enter:

```
TIME-WAIT
```

The main reasons include allowing delayed old segments to expire and ensuring the final ACK can be retransmitted if necessary.

Think:

```
Old packets
    ↓
Could still exist in network
    ↓
TIME-WAIT
    ↓
Give them time to disappear
```

Don't memorize a simplistic statement like:

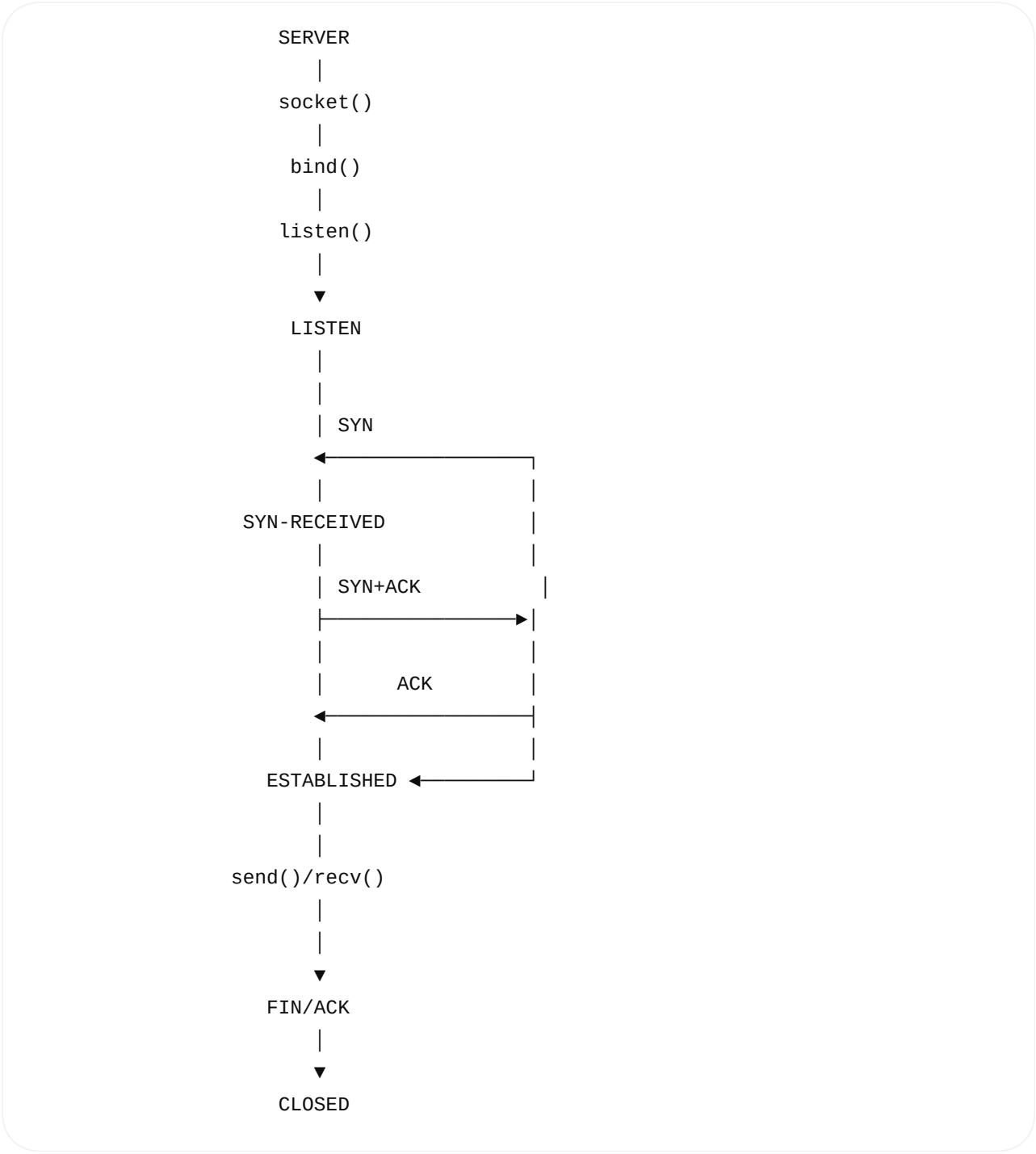
> "TIME_WAIT is just waiting for the server."

That's not correct.

---

23. Complete TCP Lifecycle

Now put the entire Day 5 together:



Client side:

```
CLOSED
   |
socket()
   |
connect()
   |
SYN-SENT
   |
   | SYN+ACK
   ▼
ESTABLISHED
   |
send()/recv()
   |
```

```
close()
   |
FIN-WAIT
   |
TIME-WAIT
   |
CLOSED
```

---

24. API vs TCP Action — Very Important

This table is worth memorizing.

Application API	What application asks for	TCP/Kernel action
`socket()`	Create socket	Creates socket object
`bind()`	Assign local address/port	Associates local endpoint
`listen()`	Accept incoming connections	Puts TCP socket into listening mode
`connect()`	Connect to remote endpoint	TCP connection establishment
`accept()`	Get incoming connection	Returns connected socket
`send()`	Send application data	TCP segments and transmits
`recv()`	Receive application data	Reads data from socket receive buffer
`close()`	Close socket	Begins/causes connection shutdown as appropriate

---

25. The Biggest Interview Concept

Don't say:

> " `connect()` is the Transport Layer."

Instead say:

> `connect()` **is a socket API/system call used by the application. For a TCP socket, the Linux kernel's TCP implementation performs the TCP connection establishment as a result of that call.**

Similarly:

> `bind()` **is a socket API that associates a socket with a local IP address and port; it does not itself establish a TCP connection.**

This distinction is very important.

---

26. Day 5 Interview Questions

Try answering these without looking back:

Q1

What is the purpose of `socket()` ?

Q2

Does `socket()` establish a TCP connection?

Q3

What does `bind()` do?

Q4

Does `bind()` send SYN?

Q5

What is the purpose of `listen()` ?

Q6

What is the difference between a listening socket and a connected socket?

Q7

What does `accept()` return?

Q8

Which API initiates TCP connection establishment from the client?

Q9

What are the three TCP handshake messages?

Q10

What does `FIN` mean?

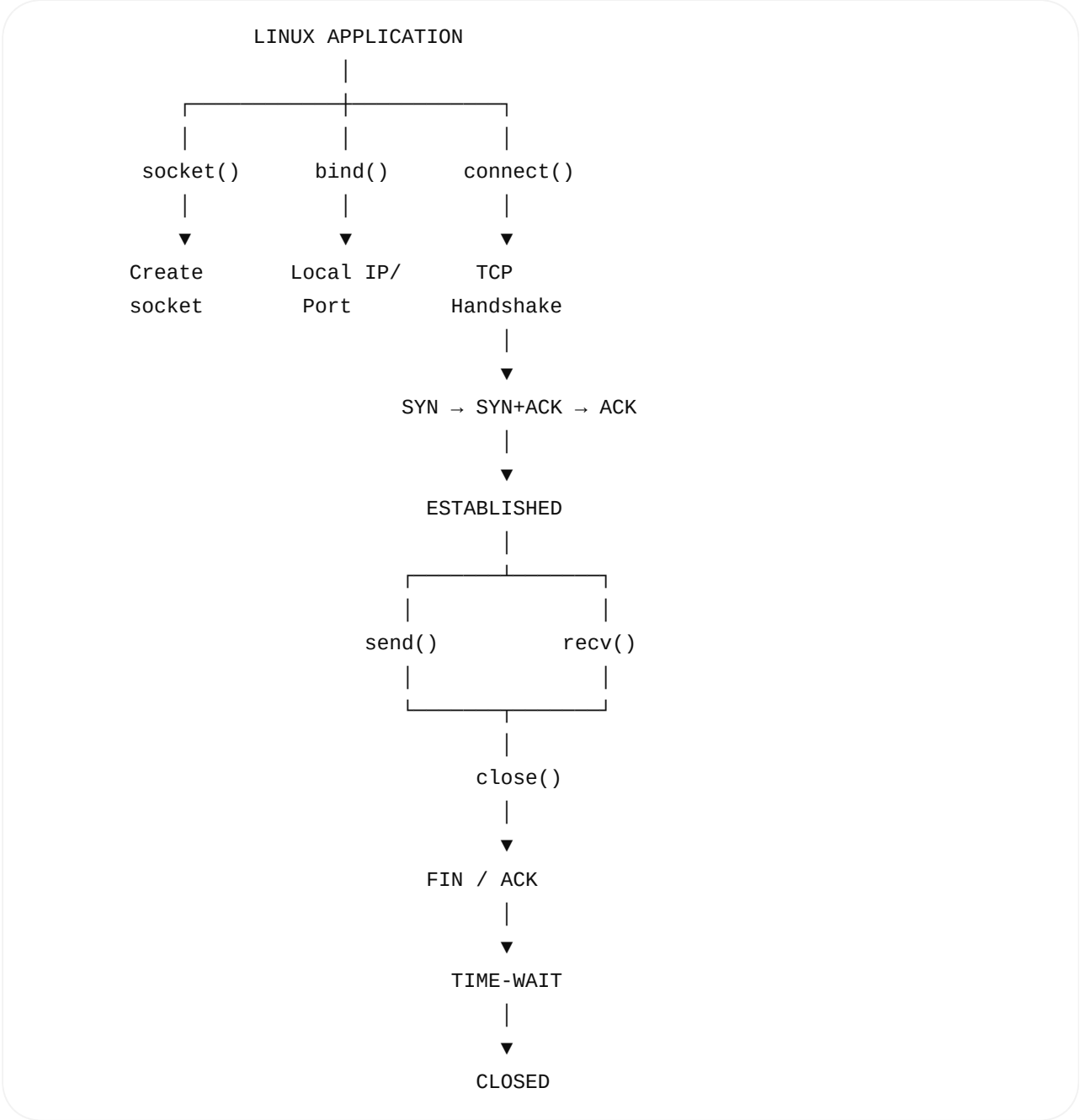
Q11

Why does TCP use `TIME-WAIT` ?

Q12

What is the difference between `close()` and the TCP FIN mechanism?

⭐ Day 5 Must-Remember Diagram



One sentence to remember:

> **Socket APIs are the application's interface to the kernel; the kernel's TCP/IP stack performs the actual networking work.**

Next Day 6: we'll move from TCP APIs into **UDP + TCP vs UDP + ports + sockets + client/server communication**, then connect that to **DNS, HTTP, HTTPS, SSH** so you can see where TCP/UDP are actually used in real applications.